Python Engineering - Quick Reference

Class 05 - Lists & Tuples: simple reference guide

Quick reference for **week_02_data_structures -> class_05_lists_tuples**. Goal: open this PDF months later and quickly remember what each structure is, how indexing works, how data can be modified, and which common operations are available.

1. List: simple definition

A **list** is like an ordered box with several compartments. Each compartment has a position (an index) and can hold almost any Python object. Lists are **mutable**: they can be changed after creation.

```
heights = [0.0, 5.0, 9.9, 14.7]
          ^     ^    ^     ^
index     0     1    2     3

heights[-1]   # last item -> 14.7
```

2. Tuple: simple definition

A **tuple** groups several values in a fixed order. It resembles a list, but its structure is **immutable**: elements cannot be freely replaced, added, or removed after creation.

```
state = (0.1, 5.0, 49.019)
#        t    h      v

t, h, v = state   # tuple unpacking
```

Important: an immutable tuple may contain mutable objects (for example, a list). The tuple keeps the same references, while the inner list can still change.

3. Indexing

Indexing means accessing an element by its position. Python starts counting at 0.

```
values = [10, 20, 30, 40]

values[0]   # 10
values[2]   # 30
values[-1]  # 40
values[-2]  # 30
```

4. Nested indexing

When one structure contains another, use one index for each nesting level.

```
# List inside a list
matrix = [[1, 2, 3],
          [4, 5, 6]]

matrix[1]      # [4, 5, 6]
matrix[1][2]   # 6

# List inside a tuple
data = ([10, 20, 30], 'sensor_A')

data[0]        # [10, 20, 30]
data[0][1]     # 20

# Tuple inside a list
trajectory = [(0.0, 0.0, 50.0),
              (0.1, 5.0, 49.019)]

trajectory[1]       # (0.1, 5.0, 49.019)
trajectory[1][1]    # 5.0
```

Mental rule

`a[i][j]` means: first get `a[i]`; then index `[j]` inside that result.

5. Slicing: extracting several elements

```
values = [10, 20, 30, 40, 50]

values[1:4]   # [20, 30, 40]
values[:3]    # [10, 20, 30]
values[2:]    # [30, 40, 50]
values[::2]   # [10, 30, 50]
```

The right endpoint is excluded. `[start:stop:step]` follows the same idea as range().

6. Basic list and tuple operations

Operation	Example	Result / meaning
Concatenation (+)	[1,2] + [3,4]	[1,2,3,4]
Repetition (*)	[1,2] * 3	[1,2,1,2,1,2]
Membership	3 in [1,2,3]	True
Length	len(values)	number of elements
Maximum	max(values)	largest comparable element
Minimum	min(values)	smallest comparable element
Numeric sum	sum(values)	sum of numeric elements
Sorted copy	sorted(values)	returns a new sorted list

Careful: multiplying a list by 3 repeats the list; it does not multiply each numeric element. Element-wise multiplication will come later with NumPy.

7. Main list methods

Method	What it does	Example
.append(x)	add one element at the end	a.append(5)
.extend(it)	add several elements	a.extend([5,6])
.insert(i,x)	insert at a position	a.insert(1,99)
.remove(x)	remove first matching value	a.remove(99)
.pop(i)	remove and return an item	a.pop(-1)
.clear()	empty the list	a.clear()
.index(x)	index of first match	a.index(5)
.count(x)	number of occurrences	a.count(5)
.sort()	sort the same list	a.sort()
.reverse()	reverse the same list	a.reverse()
.copy()	create a shallow copy	b = a.copy()

8. Main tuple methods

Tuples have far fewer methods because their structure cannot be modified.

Method / function	Use
.count(x)	number of occurrences of x
.index(x)	index of the first x
len(tuple)	number of elements
max(tuple)	maximum if elements are comparable
min(tuple)	minimum if elements are comparable

9. Modification: list yes, tuple no

```
a = [10, 20, 30]
a[1] = 99       # OK -> [10, 99, 30]

b = (10, 20, 30)
b[1] = 99       # ERROR
```

10. Mutability and references

```
a = [1, 2]
b = a

b.append(3)

print(a)  # [1, 2, 3]
print(b)  # [1, 2, 3]
```

a and b point to the same object. For an independent list use `b = a.copy()`.

11. Useful engineering patterns

```
# Time series by variable
times = []
heights = []

times.append(t)
heights.append(h)

# Time series by state
trajectory = []
trajectory.append((t, h, v))

# Maximum altitude from trajectory
max_h = max(state[1] for state in trajectory)
```

12. Which one should I choose?

Need	Structure
Collection that will change	list
Fixed state with ordered fields	tuple
History by variable	several lists
History by epoch / state	list of tuples

Mini cheat sheet

```
LIST
[]                  create
a[i]                indexing
a[i:j]              slicing
a.append(x)         add one
a.extend(xs)        add many
a[-1]               last item
len(a), max(a)      useful functions
```

```
TUPLE
()                  create
t[i]                indexing
a, b, c = t         unpack
immutable structure

NESTED
x[i][j]             level 1, then level 2
```